

FreightDesk

Transportation & Warehouse Management System
Complete build & operations guide

Prepared by your build council — Chief Information Officer & Chief Supply Chain Officer
July 2, 2026

This document explains everything that was built to complete the FreightDesk web application: the eight operating modules, the database that powers them, the security model, how to run and deploy the app, how to use each screen day to day, and a practical roadmap for operating it as the core system of a new transportation / 3PL business.

1. Executive summary

FreightDesk is now a functional, end-to-end TMS/WMS. It is a React single-page application backed directly by Supabase (Postgres database, authentication, and file storage) and deployed as a static site on Cloudflare Pages. There is no custom backend server to maintain — access control is enforced inside the database with Row Level Security, which keeps hosting costs near zero while the business ramps up.

The app started this engagement as a working skeleton: login, a dashboard, and read-only tables for shipments, customers, carriers, documents, and tasks. It is now a complete operating system for a small freight operation, with two entirely new modules (Drop Trailers and Inventory), full add/edit capability across every module, real document uploads, and an expanded dashboard.

What was delivered in this build

- **Drop Trailer system (new):** a yard board that tracks every trailer by site, spot, carrier, status, seal, and contents, with gate-in / gate-out workflow and dwell-time aging alerts.
- **Inventory tracker (new):** warehouses, a SKU catalog (optionally owned by a customer — the 3PL model), on-hand levels per warehouse, low-stock alerts, and a movement ledger (inbound / outbound / adjustment) kept consistent by an atomic database function.
- **Shipment tracker (upgraded):** inline status changes, customer and equipment columns, status filtering, and auto-stamping of the delivery timestamp when a load is marked delivered.
- **Customers (upgraded):** full add/edit forms covering contacts, address, facility type, business hours, and special instructions, plus one-click active/inactive toggling.
- **Carriers / freight providers (upgraded):** add/edit with MC and USDOT numbers, insurance details, COI expiration warnings (red within 30 days), and a compliance badge that turns green only when W-9, COI, and the carrier packet are all on file.
- **Documents (upgraded):** real file uploads to a private Supabase Storage bucket, typed as BOL, POD, invoice, rate confirmation, COI, W-9, or carrier packet, linkable to loads, with secure signed-URL downloads.
- **Tasks (upgraded):** add tasks with assignee, due date, and load link; click a status chip to advance open → in progress → done; overdue highlighting; archive/restore.
- **Dashboard (upgraded):** eight live KPIs (active shipments, on-time %, booked revenue, customers, trailers in yard, low-stock lines, open tasks, inventory lines) and two charts (load volume by month, revenue by carrier).

Critical fixes found along the way

- **Broken database URL:** the app's .env pointed at a misspelled Supabase project URL, so the deployed app could never reach the database. Fixed.
- **Stale type definitions:** the code's generated database types were far behind the live schema (the live database already had carrier compliance fields, shipper/consignee fields, and a yard_trailers table the code didn't know about). Regenerated and fully wired into the UI.
- **Status mismatch:** the Add Load form offered statuses ('quoted', 'cancelled') that the database rejected — saving those loads would fail. The database constraint was aligned with the app.

- **View security hole:** the database views used to read data bypassed Row Level Security, meaning the public API key could read operational data without a login. All views now run with the caller's permissions.

2. System architecture (CIO view)

Stack: React 18 + TypeScript + Vite + Tailwind CSS on the front end; TanStack Query for server-state caching; Supabase (Postgres 17, GoTrue auth, Storage) as the backend; Cloudflare Pages for hosting; GitHub Actions for CI (typecheck + build on every push).

Data flow: pages never call the database directly. Each module has React Query hooks (src/hooks) that call a typed Supabase client. Reads go through 'enriched' Postgres views that pre-join names and computed fields (so the UI never joins tables client-side); writes go to base tables or database functions (RPCs).

Layer	Technology	Purpose
UI	React + Tailwind (Vite)	8 module pages, modal forms, tables, charts
State	TanStack Query	Caching, refetching, optimistic invalidation
API	Supabase client (typed)	Auto-generated REST over Postgres, auth-aware
Database	Postgres 17 + RLS	11 tables, 4 views, 2 RPCs, all row-level secured
Files	Supabase Storage	Private 'documents' bucket, signed-URL downloads
Hosting	Cloudflare Pages	Static dist/ build, SPA fallback redirect
CI	GitHub Actions	tsc typecheck + production build gate on main

Key application files

```
src/pages/ Dashboard, Shipments, Trailers, Inventory, Customers,
Carriers, Documents, Tasks, Login
src/hooks/ useLoads, useTables, useYard, useInventory, useMutations
src/components/ Layout, Sidebar, DataTable, PageHeader, AddLoadForm,
ui/Modal, ui/Badge, ui/StatCard
src/lib/ supabase.ts (client), database.types.ts (generated),
types.ts (domain types), auth, distance/quote engine
supabase/migrations 0001 core, 0002 customers/docs/tasks,
0003 yard trailers, 0004 warehouse inventory + fixes
```

3. Database design

All keys are auto-generated bigints. **loads** is the hub of the transportation side; **inventory_movements** is the ledger of the warehouse side. Every table has Row Level Security enabled; the current policy grants full access to any authenticated user (right for a single-team startup — tighten to role-based policies as you hire).

Table	What it holds
carriers	Freight providers: SCAC, mode, MC/USDOT, insurance, COI expiry, W-9/COI/packet compliance flags, factoring info
customers	Shippers/receivers: contacts, address, facility type, business hours, special instructions
lanes	Origin/destination pairs with miles
loads	Shipments: ref, status, rate, weight, equipment, BOL #, commodity, full shipper & consignee blocks, pickup/ETA/delivery
yard_trailers	Drop trailer pool: site, trailer #, carrier, status (Empty/Loaded/Partial/OOS/Reserved), spot, seal, contents, gate in/out
warehouses	Facilities: code, name, address, dock doors, trailer spots
inventory_items	SKU catalog: description, owning customer (3PL), unit of measure, unit weight & value
inventory_levels	Qty on hand / allocated / reorder point per warehouse-item (unique pair)
inventory_movements	Immutable ledger: inbound, outbound, adjustment, with load ref and notes
documents	Document metadata: type, storage path, linked load
tasks	Work items: title, assignee, status, due date, load link, archive flag

Views and functions

- **loads_enriched** — loads joined to carrier, lane, and customer names plus computed miles. What the Shipments page and dashboard read.
- **yard_trailers_enriched** — trailers with carrier names, for the yard board.
- **inventory_levels_enriched** — levels with SKU, warehouse, owner, computed qty_available and a low_stock flag.
- **inventory_movements_enriched** — the ledger with SKU and warehouse codes.
- **record_inventory_movement()** — the only sanctioned way to change stock: writes the ledger row and updates the level in one atomic transaction (inbound adds, outbound subtracts, adjustment applies a signed quantity). This guarantees the ledger always explains the on-hand number.
- **add_load()** — quick-create RPC for loads (pre-existing).

All four views run with `security_invoker = on` so Row Level Security on the base tables applies to everyone who queries them — without this, the public anon key could read business data without logging in.

4. Using the app, module by module

Dashboard

Your morning screen. Top row: active shipments, on-time percentage of delivered loads, total booked revenue (all non-cancelled loads), and customer count. Second row: trailers currently in the yard, inventory lines at/below reorder point, open tasks, and total inventory lines. Below: load volume by pickup month and booked revenue by carrier (top 6).

Shipments (the TMS core)

- **Add a load:** click + Add load, enter a reference and origin/destination ZIP codes. Mileage is estimated automatically from ZIP-3 centroids and a quote is computed from your rate-per-mile, fuel %, and minimum charge inputs.
- **Track status:** change status directly in the table — quoted → booked → in transit → (delayed/exception) → delivered, or cancelled. Marking a load delivered stamps the delivery time automatically.
- **Filter:** the status dropdown in the header narrows the board to one stage.
- Columns show customer, lane, miles, carrier, equipment, rate, and on-time flag — all from one enriched view, no extra clicks.

Drop Trailers (yard board)

- **Gate in:** + Gate in trailer with site, trailer #, carrier, status, spot, seal, contents. Gate-in time is stamped automatically.
- **Live pool:** stat cards show trailers in yard and how many are Loaded, Empty, or Out of service, plus an aging count for anything sitting more than 48 hours.
- **Dwell:** every trailer in the yard shows hours since gate-in; over 48h turns red — your cue to bill detention or turn the trailer.
- **Gate out:** one click stamps the departure and removes the trailer from the active pool (history is preserved).
- **Edit** reopens the full form to reassign spots, update seals, or change status as trailers are loaded/unloaded.

Inventory (the WMS core)

- **Add item:** define a SKU, description, unit of measure, unit weight/value, and (optionally) the customer who owns the goods — this is what makes it a 3PL system: you store other companies' freight and can report by owner.
- **Record movement:** pick warehouse, item, type (inbound receipt, outbound shipment, or +/- adjustment), quantity, and an optional load reference tying warehouse activity to a shipment. On-hand updates instantly and atomically.
- **Levels table:** on hand, allocated, available, reorder point, and a red Low stock badge per warehouse-item line.
- **Ledger:** the Recent movements table is your audit trail — every unit on hand is explained by receipts minus shipments plus adjustments.

Customers

Add/edit contact info, address, facility type, business hours, and special instructions (dock notes, appointment rules). Click the Active pill to deactivate a customer without deleting history. Customer names automatically appear on shipments and inventory items linked to them.

Carriers (freight providers)

- Add/edit name, SCAC, mode (truckload / LTL / intermodal / parcel), MC and USDOT numbers, contact, and insurance company.
- **Compliance at a glance:** the Compliance badge is green only when W-9, COI, and carrier packet are all checked — don't tender freight to a yellow badge.
- **COI expiration** turns red within 30 days of expiry so you can chase renewals before they lapse.

Documents

Upload BOLs, PODs, invoices, rate cons, COIs, W-9s, and carrier packets. Files land in a private storage bucket — only logged-in users can download, via short-lived signed links. Link a document to its load so the paperwork for any shipment is one click away. (Records created before this build were metadata-only; the app will tell you if a file isn't actually in storage.)

Tasks

The team to-do list. Add tasks with assignee, due date, notes, and an optional load link. Click the status chip to advance it (open → in progress → done). Overdue dates show red. Archive finished work to keep the board clean; the Archived toggle brings it back.

5. Running, building, and deploying

Local development

```
cd 'Freight desk'
npm install # once
npm run dev # http://localhost:5173
npm run typecheck # strict TypeScript check (passes clean)
npm run build # typecheck + production bundle to dist/
npm run deploy # build, then wrangler pages deploy dist
```

The app needs `.env` with `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`. This file was fixed during the build (the URL had a typo that broke all database access). Never put a service-role key in `.env` — only the anon key, which is safe to expose because RLS does the real access control.

Sign-in

The app uses Supabase email/password auth. Create users in the Supabase dashboard under Authentication → Users (Add user). Anyone without an account sees only the login screen and can read nothing.

What was verified

- Strict TypeScript typecheck passes with zero errors (this is the repo's CI gate).
- All migrations applied to the live database; every view and the movement RPC verified with live queries.
- Seed data loaded: 2 warehouses, 8 SKUs, 16 stock lines, 19 ledger movements, 7 yard trailers (5 new), plus the existing 10 carriers, 6 customers, and 19 loads.
- Storage bucket 'documents' created with authenticated-only read/write policies.
- Views converted to security-invoker so the anon key cannot bypass RLS.

Note: the final Vite bundling step should be run once on your Mac (`npm run build`) before deploying — the build sandbox used for this work couldn't execute the platform-specific bundler binary, though the full compiler check it gates on passes cleanly. GitHub CI runs the same build on every push as a second net.

6. Operating roadmap for your 3PL (CSCO view)

Use it like this from day one

- **Quote → book:** create the load at 'quoted' with the auto-quote; flip to 'booked' when the customer accepts and assign the carrier.
- **Dispatch:** mark in_transit at pickup; use delayed/exception honestly — your on-time % is only as good as your discipline here.
- **Yard cadence:** walk the yard (or check the board) twice daily; anything red on dwell is money — bill detention or turn it.
- **Warehouse cadence:** record movements as they happen, not at end of day; reconcile low-stock lines with owners weekly.
- **Paper discipline:** upload the POD the day of delivery and invoice the same day — cash flow is the #1 killer of young brokerages.
- **Carrier onboarding:** no green compliance badge, no freight. COI lapses are an uninsured-loss risk you cannot absorb.

Suggested next builds (in priority order)

- **Role-based access:** split RLS policies into dispatcher / warehouse / accounting roles before the first hire.
- **Invoicing & settlements:** customer invoice and carrier-pay fields on loads, an aging view, and margin per load (rate to customer minus rate to carrier).
- **Allocation workflow:** reserve inventory (qty_allocated) against outbound orders instead of tracking it manually.
- **Customer portal:** read-only tracking + inventory views per customer (RLS makes this straightforward).
- **Notifications:** email/SMS on status changes, COI expirations, low stock, and 48h trailer dwell.
- **EDI/API integrations:** load boards (DAT/Truckstop) and customer EDI 204/214/210 as volume justifies it.

Compliance notes for a new transportation company

Before moving freight commercially you will need operating authority (MC number) from FMCSA, a USDOT number, a BOC-3 process agent filing, and — as a broker — a \$75,000 surety bond (BMC-84/85), plus insurance appropriate to your model. The Carriers module already tracks the same credentials for the carriers you hire. This guide is operational, not legal advice — confirm requirements with a transportation attorney or compliance service.

Bottom line: FreightDesk now covers the full quote-to-delivery lifecycle on the transportation side and receipt-to-shipment on the warehouse side, with a clean, secured data model underneath. It is ready to run a real book of business — and structured so each next capability is an addition, not a rewrite.